

# ENERGY-EFFICIENT AND COMMUNICATION-AWARE ARCHITECTURES FOR ACCELERATING DEEP LEARNING WORKLOADS

CSQ413 Seminar Report

Submitted by:

LMDL22CS206 22CSA71 Parvathy Prasad

B. Tech. in Computer Science & Engineering



**GOVT. MODEL ENGINEERING COLLEGE**

Department of Computer Engineering

Thrikkakara, Kochi - 682021

October 2025

**MODEL ENGINEERING COLLEGE,  
THRIKKAKARA  
DEPARTMENT OF COMPUTER ENGINEERING**



**CERTIFICATE**

This is to certify that the report titled ***Energy-Efficient and Communication-Aware Architectures for Accelerating Deep Learning Workloads*** is a bonafide record of the CSQ413 seminar presented by **LMDL22CS206 22CSA71 Parvathy Prasad** B. Tech. (Seventh Semester), Department of Computer Engineering

Guide

Mrs Thasni Ebrahim  
Assistant Professor  
Computer Engineering

Coordinator

Mr Murali Mohanan  
Associate Professor  
Computer Engineering

Head of the Department

Dr Binu V P  
Professor  
Department of Computer Engineering

**October 2025**

---

## ACKNOWLEDGMENTS

I express my sincere gratitude to my seminar guide, **Mrs. Thasni Ibrahim**, Assistant Professor, Department of Computer Engineering, Model Engineering College, for her valuable support and expert guidance throughout this work.

I extend my appreciation to the seminar coordinator, **Associate Professor Mr. Murali Mohanan**, for his assistance and supervision during the course of this seminar.

My heartfelt thanks go to **Prof. Dr. Jayachandran E. S.**, Principal of Model Engineering College, and **Prof. Dr. Binu V. P.**, Head of the Department of Computer Engineering, for providing the necessary facilities and encouragement.

**Parvathy Prasad**

## Abstract

**Abstract:** Deep Neural Networks (DNNs) have become the backbone of modern Artificial Intelligence (AI) systems, enabling applications in image recognition, natural language processing, and autonomous systems. However, as DNN models grow in depth and complexity, they impose substantial computational and memory demands that challenge the efficiency of traditional hardware. The high cost of data movement between memory and compute units has made *energy-efficient* and *communication-aware* architectures critical for future AI acceleration.

Energy-efficient and communication-aware architectures for accelerating deep learning workloads examines recent advances in hardware/software co-design for efficient deep learning acceleration, covering model-level optimizations such as pruning, quantization, and sparsity-aware training, alongside hardware-level innovations including systolic arrays, near-memory computing, and 2.5D chiplet-based architectures connected via Network-on-Package (NoP). A comparative study of representative designs like CANNON, HyGCN, and SpAtten highlights how codesign across algorithmic and architectural domains can achieve significant improvements in energy efficiency, scalability, and performance-per-watt.

**Keywords:** Deep Learning Acceleration, Energy Efficiency, Sparsity, Hardware/Software Co-design, Chiplet Architecture, Network-on-Package (NoP).

# Contents

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                              | <b>4</b>  |
| <b>2</b> | <b>Background and Motivation</b>                                 | <b>5</b>  |
| 2.1      | Why communication matters . . . . .                              | 5         |
| 2.2      | Key concepts . . . . .                                           | 5         |
| <b>3</b> | <b>Related Work</b>                                              | <b>7</b>  |
| 3.1      | Sparsity-aware accelerators . . . . .                            | 7         |
| 3.2      | Chiplet and 2.5D integration . . . . .                           | 7         |
| <b>4</b> | <b>Sparsity-Aware Hardware Acceleration</b>                      | <b>8</b>  |
| 4.1      | Pruning taxonomy and patterns . . . . .                          | 8         |
| 4.2      | Sparsity-aware training . . . . .                                | 9         |
| 4.3      | Specialized accelerators for GNNs and Transformers . . . . .     | 9         |
| <b>5</b> | <b>Multi-Chiplet (2.5D) Architectures</b>                        | <b>10</b> |
| 5.1      | Why chiplets? . . . . .                                          | 10        |
| 5.2      | Network-on-Package (NoP) and interconnect technologies . . . . . | 10        |
| 5.3      | Mapping DNN layers to chiplets . . . . .                         | 11        |
| <b>6</b> | <b>Design Considerations and Practical Guidelines</b>            | <b>12</b> |
| 6.1      | Model-side (software) recommendations . . . . .                  | 12        |
| 6.2      | Hardware-side (architecture) recommendations . . . . .           | 12        |
| 6.3      | Evaluation metrics . . . . .                                     | 12        |
| <b>7</b> | <b>Case Studies and Comparative Analysis</b>                     | <b>13</b> |
| <b>8</b> | <b>Tools, Simulation Frameworks and Methodology</b>              | <b>15</b> |
| <b>9</b> | <b>Conclusions and Future Work</b>                               | <b>16</b> |
|          | <b>REFERENCES</b>                                                | <b>16</b> |

# List of Figures

|     |                                                                                                                                                      |    |
|-----|------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1 | Taxonomy of research directions for energy-efficient DL acceleration (model compression, dataflow/architecture, hardware technologies, packaging). . | 6  |
| 4.1 | Pruning examples: (a) CNN structured/unstructured, (b) GNN node/edge pruning, (c) Transformer head and token pruning. . . . .                        | 8  |
| 5.1 | Monolithic vs 2.5D chiplet-based architecture with NoP (conceptual). . .                                                                             | 11 |
| 7.1 | Qualitative comparison of architectural approaches (conceptual tradeoff chart). . . . .                                                              | 14 |

# List of Tables

|     |                                                                  |    |
|-----|------------------------------------------------------------------|----|
| 5.1 | Representative chiplet-based DNN research (focus areas). . . . . | 11 |
|-----|------------------------------------------------------------------|----|

# Chapter 1

## Introduction

Modern AI workloads put massive strain on computational and memory subsystems. Energy used for moving data often exceeds that used for arithmetic operations, making *communication-aware* design as important as compute optimization. This seminar explores two complementary directions: (1) model and algorithmic techniques (sparsity, pruning, quantization) that reduce computation and data movement, and (2) hardware and packaging approaches (near-memory compute, in-memory computing, chiplet/2.5D integration) that minimize energy per operation and improve scalability. A detailed discussion and examples are provided in the chapters that follow.

# Chapter 2

## Background and Motivation

### 2.1 Why communication matters

Data movement is expensive: moving a word from DRAM to a compute unit can cost orders of magnitude more energy than a single integer or floating-point multiply-accumulate. As models expand, the relative cost of communication grows. Chip designers therefore must co-optimize compute, memory, and interconnect. See the high-level taxonomy in Figure 2.1.

### 2.2 Key concepts

- **Pruning and Sparsity:** Removing redundant weights/activations to reduce memory and computation.
- **Quantization:** Reducing numeric precision to shrink memory and improve energy per operation.
- **In-memory / Near-memory computing:** Bringing compute closer to storage to reduce movement.
- **Chiplet/2.5D design:** Packaging multiple small dies (chiplets) to manage yield and scale while enabling heterogeneous integration.
- **Lightweight Architectures (e.g., MobileNets, EfficientNets):** Designing intrinsically energy-efficient models using techniques like depthwise separable convolutions to reduce parameters and FLOPS from the ground up.
- **Knowledge Distillation:** Transferring knowledge from a large, high-performing "teacher" model to a smaller, more efficient "student" model, allowing the student to maintain high accuracy with lower computational cost.

- **Specialized DL Accelerators (e.g., TPUs, NPU):** Utilizing custom hardware, often featuring systolic arrays or massive parallelism, specifically designed to accelerate matrix operations and data flow for deep learning with far greater energy efficiency than general-purpose CPUs/GPUs.

| Model Compression                                                                 | Dataflow and Architecture                                                                                                                                                                             | Hardware Technologies                                                                                                                  | Packaging and Integration Technologies                          |
|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| Quantization<br>Sparsification (Pruning)<br>Compact Model<br>Tensor Decomposition | <b>Dataflow</b><br>Weight Stationary<br>Input Stationary<br>Output Stationary<br>Row Stationary<br>No Local Reuse<br><br><b>Architecture</b><br>In-Memory Computing<br>Systolic Array<br>CGRA<br>FPGA | CMOS<br><br><b>Near-Memory Computing</b><br>DRAM<br><br><b>In-Memory Computing</b><br>Resistive RAM<br>SRAM<br>PCM<br>STT-RAM<br>CBRAM | Monolithic<br>2.5D Architectures (Chiplets)<br>3D Architectures |

Figure 2.1: Taxonomy of research directions for energy-efficient DL acceleration (model compression, dataflow/architecture, hardware technologies, packaging).

# Chapter 3

## Related Work

This section summarizes notable prior works in (i) sparsity-aware accelerators, (ii) GNN and transformer accelerators, and (iii) chiplet-based multi-die accelerators.

### 3.1 Sparsity-aware accelerators

Representative works include SNAP, TensorDash, and TensorArray techniques that handle unstructured and structured sparsity through architectural and scheduler optimizations. CANNON is a framework emphasizing communication-aware dynamic sparse training to reduce communication and energy during distributed training and inference. (See the discussion and benchmarks in the source paper.) :contentReference[oaicite:3]index=3

### 3.2 Chiplet and 2.5D integration

Simba, SIAM, NN-Baton, SWAP, Floret and Gemini present diverse approaches to scaling DNN inference and training across chiplets using network-on-interposer/noP topologies and mapping algorithms. Heterogeneous chiplet designs (big-little) have been shown to yield better overall energy-delay-area product. :contentReference[oaicite:4]index=4

# Chapter 4

## Sparsity-Aware Hardware Acceleration

### 4.1 Pruning taxonomy and patterns

Pruning techniques fall into:

- **Unstructured pruning:** fine-grained weight removal, best compression but poor hardware regularity.
- **Structured pruning:** channel/filter-level removal enabling regular compute patterns.
- **N:M (hybrid) sparsity:** e.g., Nvidia’s N:M pattern providing a compromise.

(Examples and figures adapted from the uploaded paper.) :contentReference[oaicite:5]index=5

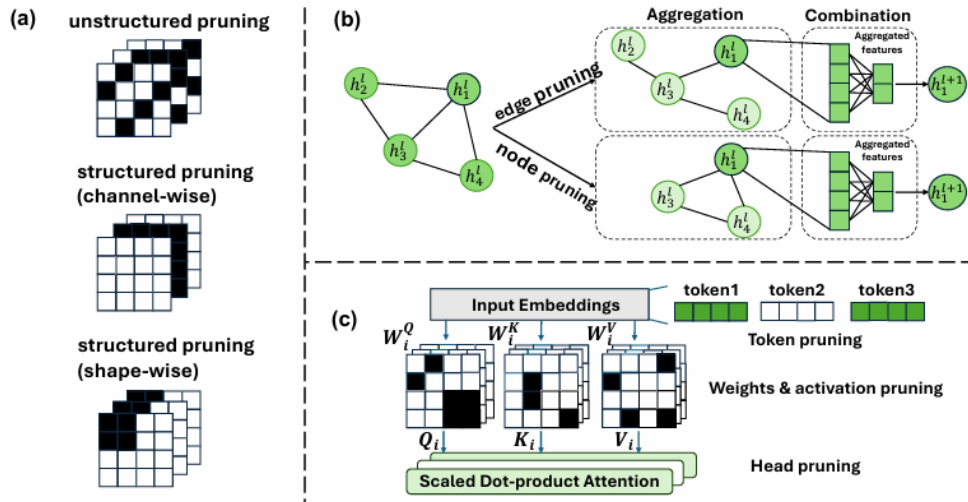


Figure 4.1: Pruning examples: (a) CNN structured/unstructured, (b) GNN node/edge pruning, (c) Transformer head and token pruning.

## 4.2 Sparsity-aware training

CANNON dynamically prunes and regrows weights based on both weight importance and communication cost — an approach that balances model size with communication overhead in distributed hardware. Reported benefits include substantial reductions in communication latency and energy-delay product. :contentReference[oaicite:6]index=6

## 4.3 Specialized accelerators for GNNs and Transformers

- **HyGCN / AWG-GCN / VersaGNN:** Engines that split aggregation and combination, use SIMD for irregular aggregation, and balance workload dynamically to get high PE utilization. :contentReference[oaicite:7]index=7
- **SpAtten / ViTCoD:** Architectures that prune attention heads and tokens and use quantization to speed up transformer workloads. :contentReference[oaicite:8]index=8
- **Knowledge Distillation:** Training a compact student model to mimic the output of a large, complex teacher model, retaining accuracy with far fewer parameters and reduced inference energy.
- **Systolic Arrays:** A highly regular, grid-like arrangement of Processing Elements (PEs) that enables massive parallel computation and efficient, localized data reuse, minimizing external memory access.
- **Low-Precision/Binary Neural Networks (BNNs):** Extreme quantization where weights/activations are restricted to 1 or 2 bits, drastically reducing the size of the model and replacing expensive multiplications with low-cost bitwise operations (XNOR).

# Chapter 5

## Multi-Chiplet (2.5D) Architectures

### 5.1 Why chiplets?

Monolithic chips suffer yield and cost problems as die area increases. Chiplets allow modularity, heterogeneous integration, and better yield economics. They also enable specialization (e.g., IMC chiplets + general-purpose chiplets) to match non-uniform DNN layer characteristics. :contentReference[oaicite:9]index=9

### 5.2 Network-on-Package (NoP) and interconnect technologies

NoP choices include:

- **Silicon interposers:** very high density; used for bandwidth-hungry designs.
- **EMIB / bridge-based interconnects:** lower cost, adequate bandwidth for mid-range designs.
- **Photonic interconnects:** promising for ultra-high-speed, energy-efficient inter-chip links.

(References and technical discussion adapted from the uploaded paper.) :contentReference[oaicite:10]index=10

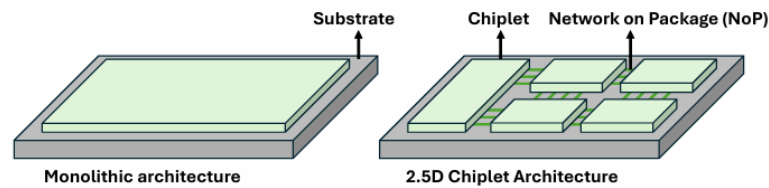


Fig. 3: 2.5D architectures combine small chips, called chiplets, connected by a network-on-package (NoP).

Figure 5.1: Monolithic vs 2.5D chiplet-based architecture with NoP (conceptual).

## 5.3 Mapping DNN layers to chiplets

A good mapping minimizes high-volume activation transfers across chiplets and places weight-heavy layers where local memory/compute are abundant. Tools and frameworks (e.g., Gemini, SIAM) propose encoding formats and mapping heuristics to automate this exploration. Table 5.1 summarizes representative works.

Table 5.1: Representative chiplet-based DNN research (focus areas).

| Work     | Focus / Contribution                        | Notes                                 |
|----------|---------------------------------------------|---------------------------------------|
| Simba    | Multi-chip-module accelerator for inference | High TOPS; tiling optimizations.      |
| SIAM     | Scalable IMC evaluation framework           | Supports array of DNN configs.        |
| NN-Baton | Workload orchestration for chiplets         | Allocation / granularity exploration. |
| SWAP     | NoP optimization for server-scale PIM       | Communication-aware mapping.          |
| Floret   | Dataflow-aware NoP for datacenter CNNs      | Space-filling-curve mapping.          |
| Gemini   | Mapping + co-exploration for chiplets       | Layer-pipeline spatial mappings.      |

# Chapter 6

## Design Considerations and Practical Guidelines

This chapter provides practical guidelines when designing or evaluating accelerators.

### 6.1 Model-side (software) recommendations

- Apply structured pruning when targeting systolic arrays / dense compute.
- Use N:M sparsity for accelerator-friendly sparsity on GPUs / tensor cores.
- Combine pruning with quantization-aware training to reduce accuracy loss.

### 6.2 Hardware-side (architecture) recommendations

- Match dataflows (weight-stationary, row-stationary, output-stationary) to layer types.
- Use in-memory compute (IMC) chiplets for weight-heavy layers to reduce movement.
- Design NoP topology (mesh, SFC, ring) based on expected data traffic patterns; co-design mapping algorithms to the NoP.

### 6.3 Evaluation metrics

Use the following for comparative evaluation:

- **Throughput** (TOPS or images/sec).
- **Energy per inference / training step**.
- **Energy-Delay-Area Product (EDAP)** for hardware-level comparisons.
- **Communication volume and latency** across chiplets.

# Chapter 7

## Case Studies and Comparative Analysis

We summarize the relative strengths of approaches:

- **Systolic / dense arrays:** Excellent for dense convolutional workloads; simpler to program but inefficient for unstructured sparsity.
- **IMC architectures:** Extremely attractive for weight-heavy layers and large matrix multiplies; require careful analog/digital tradeoffs.
- **Chiplet-based multi-die:** Best for scaling with yield and heterogeneous integration; mapping and NoP design are the key challenges.
- **Sparsity-aware accelerators:** Very effective where high sparsity can be exploited — but need scheduler and memory support for irregular accesses.
- **Low-Precision Architectures:** Extremely energy and memory efficient by using 8-bit or lower precision; however, they require careful calibration and can suffer **\*\*accuracy loss\*\*** depending on the model.
- **Knowledge Distillation:** Achieves near Teacher-model accuracy on small Student models, but requires a complex, two-stage **\*\*training process\*\*** and is most effective for classification tasks.
- **Specialized Small-Scale Accelerators (e.g., FPGAs/Edge ASICs):** Offers high Watt-per-Operation efficiency for a fixed model, but provides **\*\*low flexibility\*\*** and high Non-Recurring Engineering (NRE) costs for design.

Figure 7.1 is a conceptual mapping of the tradeoffs (performance, energy, scalability, implementation complexity).

Qualitative comparison of architectural approaches  
(conceptual tradeoff chart)

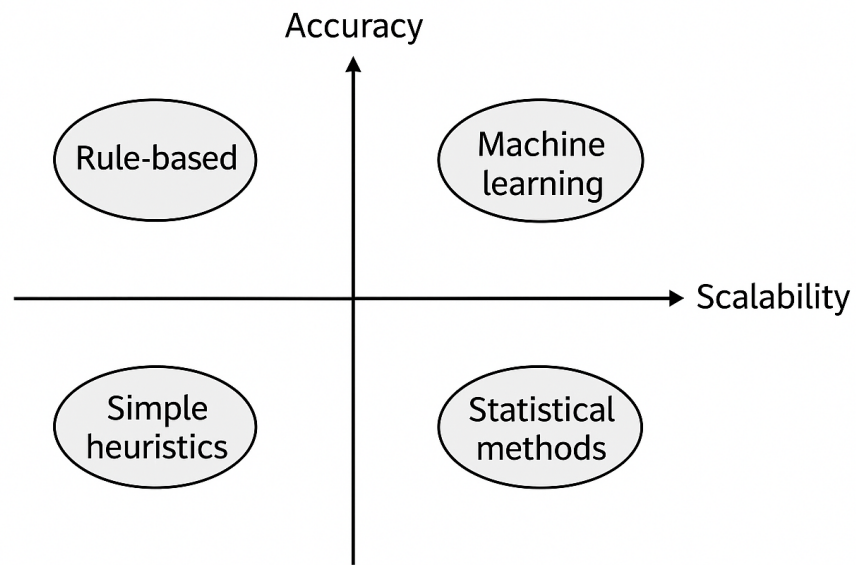


Figure 7.1: Qualitative comparison of architectural approaches (conceptual tradeoff chart).

# Chapter 8

## Tools, Simulation Frameworks and Methodology

To explore and evaluate chiplet-based accelerators and mapping strategies researchers commonly use:

- **SIAM / MuchiSim:** simulation frameworks for chiplet IMC evaluation. :contentReference[oaicite:12]index=12
- **Gemini / Opt-gen:** mapping and architecture exploration tools for co-design. :contentReference[oaicite:13]index=13
- **Cycle-accurate or NoC simulators:** for measuring communication patterns and latency.

### Recommended evaluation flow:

1. Profile model-level compute/activation/weight distributions.
2. Choose target chiplet topology and NoP technology.
3. Use mapping tool to map layers to chiplets and estimate communication.
4. Simulate with traffic/noise models, collect EDAP / throughput / latency metrics.
5. Iterate: adjust pruning/quantization and remap to improve metrics.

# Chapter 9

## Conclusions and Future Work

Energy-efficient and communication-aware architectures for accelerating deep learning workloads summarized design directions for energy-efficient and communication-aware acceleration of deep learning workloads. Key takeaways:

- **Sparsity and model compression** reduce compute and memory footprint; structured sparsity is preferred for hardware efficiency. :contentReference[oaicite:14]index=14
- **Chiplet-based 2.5D integration** offers scalability and yield advantages and enables heterogeneous specialization — mapping and NoP design remain central challenges. :contentReference[oaicite:15]index=15
- **Codesign** across model, compiler, architecture, and packaging layers is essential for best results.

**Future directions:** (i) Automatic compilers that map sparse models to chiplet topologies; (ii) photonic NoP evaluations for datacenter-scale chiplets; (iii) hardware-software co-optimization for adaptive sparsity during runtime.

# REFERENCES

- [1] A. A. Goksoy, J. Lin, U. Y. Ogras, “Energy-Efficient and Communication-Aware Architectures for Accelerating Deep Learning Workloads,” IEEE SOCC, 2024.
- [2] J.-F. Zhang et al., “SNAP: An efficient sparse neural acceleration processor,” IEEE JSSC, 2020.
- [3] M. Mahmoud et al., “TensorDash: Exploiting sparsity to accelerate DNN training,” MICRO 2020.
- [4] Y. S. Shao et al., “Simba: Scaling deep-learning inference with multi-chip-module-based architecture,” MICRO 2019.